

COMPETENCIA

Diseña aplicaciones informáticas para diferentes plataformas nativas, web e híbridas para dispositivos móviles y de escritorio, como resultado del análisis de problemas, utilizando estructuras algorítmicas basadas en diversos enfoques de programación y en tecnologías emergentes (Computación en la nube, DreamWorks para desarrollo móvil) adoptadas en el mercado nacional e internacional.

Implementa planes de administración de redes locales, extensas o en la nube, para garantizar el correcto funcionamiento de los servicios y las comunicaciones de la empresa, aplicando normas y estándares de seguridad informática.

INDICADORES DE LOGRO

- Crea aplicaciones basadas en tecnologías emergentes (Computación en la nube, Frameworks para desarrollo), atendiendo los requerimientos de los usuarios y los estándares relacionados con el desarrollo de software.
- Documenta todas las etapas del desarrollo de las aplicaciones web e híbridas para dispositivos móviles y de escritorio, basadas en tecnologías emergentes: levantamiento de requerimientos, análisis, diseño, construcción, prueba e implementación de una aplicación teniendo en cuenta todas las etapas de ingeniería de software.
- Instala servicios de red en arquitecturas de plataformas libres y propietarias para establecer comunicación entre las aplicaciones y los servicios requeridos, aplicando normas y estándares de seguridad informática.

FECHAS IMPORTANTES

Mes	Actividades y fechas para entregar	Porcentaje de evaluación	Tiempo estimado para el desarrollo de la actividad
Julio/Agosto	- Desarrollo de la unidad 1: 20 de julio al 22 de agosto. - Encuentro de bienvenida: 24 de julio.		
Agosto/Septiembre	Desarrollo de la unidad 2: 24 de agosto al 19 de septiembre.		
Septiembre/Octubre	- Desarrollo de la unidad 3: 21 de septiembre al 17 de octubre. - Entrega Etapa 1 del proyecto: 12 al 17 de octubre.	30%	5 días
Octubre/Noviembre	- Desarrollo de la unidad 4: 19 de octubre al 07 de noviembre. - Entrega Etapa 2 del proyecto: 03 al 07 de noviembre.	30%	5 días
Noviembre	- Desarrollo de la unidad 5: 09 de noviembre al 28 de noviembre. - Entrega Etapa 3 del proyecto: 23 al 28 de noviembre.	40%	5 días

DISEÑO DE EXPERIENCIA DE APRENDIZAJE POR PROYECTO

Etapas 1: Circuitos de interfaz analógica y digital

Se entrega a cada estudiante una tabla de verdad de tres entradas, que representa el estado de tres sensores digitales o discretizados. El estudiante debe:

1. Derivar la función lógica de salida a partir de la tabla de verdad (por álgebra booleana o mapas de Karnaugh).
2. Diseñar el circuito combinacional con compuertas lógicas que implemente esa función.
3. Acoplar la salida del circuito digital a un actuador analógico, resolviendo el problema de interfaz: la compuerta lógica no puede excitar directamente al motor, por lo que se requiere una etapa de interfaz con transistor (BJT como interruptor), resistencia de base y, si aplica, diodo de protección (flyback).
4. Simular y validar el circuito completo en Tinkercad, demostrando que responde correctamente a las combinaciones de entrada.
5. Entregar un informe breve que documente: la derivación de la función lógica, el esquema del circuito, y la justificación de los valores de corriente, voltaje y resistencia usados en la etapa de interfaz.

Etapas 2: Programación del prototipo con Arduino

Partiendo del circuito de la Etapa 1 (o una versión ampliada del mismo), el estudiante debe reemplazar la lógica combinacional discreta por un microcontrolador Arduino, programando en C/C++ para leer los sensores y controlar los actuadores con la misma lógica de comportamiento (o una extendida). Se espera que el estudiante:

1. Migre el circuito a una versión con Arduino en Tinkercad.
2. Programe la lectura de sensores (digital y/o analógica) y el control de actuadores.
3. Use librerías apropiadas cuando el sensor o actuador lo requiera (en lugar de reinventar el manejo de bajo nivel).
4. Escriba código legible, modular y eficiente (uso de funciones, constantes, comentarios, buenas prácticas de control de flujo).
5. Documente brevemente qué librerías usó y por qué.

Etapa 3: Implementación con ESP32 y Blynk

El estudiante migra su prototipo de Arduino a ESP32, simulado en Wokwi, y conecta el sistema a Blynk para visualizar los datos de los sensores (y, si aplica, controlar actuadores) desde un dashboard. Se espera que el estudiante:

1. Identifique y aplique las diferencias relevantes entre ESP32 y Arduino (conectividad WiFi/Bluetooth integrada, niveles de voltaje de los pines, capacidad de procesamiento, IDE/configuración de placa).
2. Configure el proyecto en Wokwi con conexión WiFi virtual.
3. Configure un dashboard en Blynk (datastreams, widgets de visualización y/o control).
4. Explique cómo se comunica Blynk (arquitectura cliente–servidor, autenticación por token/plantilla, y el protocolo que usa para el intercambio de datos entre el dispositivo y la nube).
5. Logre la visualización en tiempo real de los datos del sensor en el dashboard de Blynk desde la simulación en Wokwi.

RÚBRICAS DE EVALUACIÓN

Etapa 1 - Circuitos de interfaz analógica y digital (30%)

Criterios de evaluación	Inicial 0.00 – 3.99	Básico 4.00 – 5.99	Competente 6.00 – 7.99	Destacado 8.00 – 8.99	Estratégico 9.00 – 10.00
1. Derivación de la función lógica 20%	No deriva la función o la deriva incorrectamente a partir de la tabla de verdad.	Deriva la función con errores parciales o sin simplificar.	Deriva correctamente la función y la simplifica de forma básica.	Deriva y simplifica correctamente, justificando el método usado (K-map o álgebra booleana).	Simplifica de forma óptima y justifica decisiones de diseño (ej. minimizar compuertas o costo del circuito).
2. Diseño e implementación del circuito combinacional 25%	No implementa el circuito o no corresponde a la función derivada.	Implementa el circuito con errores de conexión o compuertas incorrectas.	Implementa correctamente el circuito con las compuertas adecuadas.	Implementa el circuito de forma limpia, ordenada y con nomenclatura clara.	Implementa el circuito de forma óptima y eficiente, con justificación técnica de las compuertas elegidas.
3. Circuitos de interfaz analógica/digital (transistor, acople sensor-actuador) 25%	No hay etapa de interfaz o el actuador no responde a la lógica digital.	Interfaz presente pero mal dimensionada (falla en algunos casos).	Interfaz funcional con transistor correctamente configurado como interruptor.	Interfaz robusta, incluye protección (ej. diodo flyback) y resistencias bien calculadas.	Interfaz óptima, con justificación técnica completa de cada componente y su dimensionamiento.

4. Conceptos básicos (corriente, voltaje, resistencia, Ley de Ohm) 15%	No demuestra comprensión de los conceptos.	Identifica los conceptos, pero no los aplica correctamente a su circuito.	Aplica correctamente los conceptos a su propio circuito.	Aplica los conceptos con cálculos explícitos y coherentes.	Aplica los conceptos con cálculos precisos y los usa para justificar decisiones de diseño.
5. Funcionamiento correcto en simulación 15%	El circuito no funciona.	Funciona parcialmente, con errores en algunas combinaciones de entrada.	Funciona correctamente para todas las combinaciones de la tabla de verdad	Funciona correctamente y de forma estable/repetible.	Funciona correctamente y el estudiante demuestra haberlo probado sistemáticamente (todas las combinaciones documentadas).
Logros:			Aspectos para mejorar:		

Etapa 2 – Programación del prototipo con Arduino (30%)

Criterios de evaluación	Inicial 0.00 – 3.99	Básico 4.00 – 5.99	Competente 6.00 – 7.99	Destacado 8.00 – 8.99	Estratégico 9.00 – 10.00
1. Eficiencia y calidad del código Arduino 30%	El código no compila o no tiene relación con la lógica requerida.	Código funcional pero desordenado, sin modularidad ni comentarios.	Código correcto, con estructura básica (funciones, nombres claros).	Código eficiente, modular, comentado y con buenas prácticas de estilo.	Código eficiente y optimizado (ej. uso adecuado de tiempos no bloqueantes, manejo de estados), con justificación técnica de las decisiones tomadas.
2. Uso apropiado de librerías 20%	No usa librerías donde eran necesarias, o las usa incorrectamente.	Usa librerías, pero de forma incompleta o con errores de configuración.	Usa correctamente las librerías necesarias para sensores/actuadores.	Usa las librerías de forma correcta y eficiente, aprovechando sus funciones.	Usa las librerías de forma óptima y puede justificar por qué eligió una librería sobre otra o cuándo prescindir de ellas.

3.Lectura de sensores y control de actuadores (lógica de programación) 20%	La lógica de lectura/control no corresponde al comportamiento esperado.	Lógica parcialmente correcta, con errores en casos específicos.	Lógica correcta y equivalente a la especificación original.	Lógica correcta y mejorada respecto a la Etapa 1 (ej. mayor robustez o rango de valores).	Lógica correcta, robusta y con manejo de casos límite o errores de lectura.
4.Circuitos de interfaz (revisión de seguimiento) 15%	El circuito de interfaz ya no funciona tras la migración a Arduino.	Interfaz funcional, pero con ajustes incorrectos al migrar a Arduino.	Interfaz correctamente adaptada al control por Arduino (niveles de voltaje, PWM si aplica).	Interfaz adaptada correctamente y optimizada para el control por microcontrolador.	Interfaz adaptada de forma óptima, con justificación técnica de los ajustes realizados respecto a la Etapa 1.
5.Funcionamiento correcto del prototipo en Tinkercad 15%	El prototipo no funciona.	Funciona parcialmente o de forma inestable.	Funciona correctamente en condiciones normales de prueba.	Funciona correctamente y de forma estable en distintos escenarios de prueba.	Funciona correctamente, de forma estable, y el estudiante demuestra pruebas sistemáticas documentadas.
Logros:			Aspectos para mejorar:		

Etapa 3 – Implementación con ESP32 y Blynk (40%)

Criterios de evaluación	Inicial 0.00 – 3.99	Básico 4.00 – 5.99	Competente 6.00 – 7.99	Destacado 8.00 – 8.99	Estratégico 9.00 – 10.00
1.Diferencias ESP32 vs Arduino aplicadas 20%	No identifica diferencias relevantes ni las aplica en el diseño.	Identifica algunas diferencias, pero no las aplica correctamente (ej. niveles de voltaje mal manejados).	Identifica y aplica correctamente las diferencias principales (voltajes, conectividad).	Identifica y aplica las diferencias con justificación técnica clara.	Aplica las diferencias de forma óptima y explica su impacto en el diseño general del prototipo IoT.
2.Configuración del dashboard de Blynk 20%	No hay dashboard funcional.	Dashboard configurado, pero con widgets/datastreams mal enlazados.	Dashboard funcional con widgets correctamente enlazados a los datastreams.	Dashboard bien diseñado, claro y con visualización adecuada al tipo de dato.	Dashboard optimizado, con visualización clara, bien organizada y justificación de las decisiones de diseño de UI.

3.Comprensión de los protocolos de Blynk 20%	No explica cómo funciona Blynk ni su comunicación.	Explicación superficial o con errores conceptuales sobre la comunicación Blynk.	Explica correctamente la arquitectura cliente–servidor y el rol de la autenticación (token).	Explica con precisión el protocolo/mecanismo de comunicación entre dispositivo y servidor de Blynk.	Explica con precisión técnica el protocolo de Blynk y lo contrasta con otros protocolos IoT comunes (ej. MQTT, HTTP).
4.Conexión ESP32-Blynk mediante Wokwi 20%	No logra establecer la conexión.	Conexión inestable o parcialmente funcional.	Conexión establecida y funcional de forma básica.	Conexión estable, con manejo correcto de credenciales y librerías.	Conexión estable, robusta y con manejo de errores de conexión (ej. reconexión ante pérdida de WiFi).
5.Funcionamiento correcto del sistema completo 20%	El sistema no funciona de extremo a extremo.	Funciona parcialmente (ej. lee sensores, pero no visualiza en Blynk, o viceversa).	El sistema funciona de extremo a extremo: lee sensores y visualiza en Blynk en tiempo real	Funciona de forma estable y con datos coherentes en el dashboard.	Funciona de forma estable, con datos coherentes, y el estudiante demuestra pruebas documentadas del sistema completo.
Logros:			Aspectos para mejorar:		